Artificial Intelligence 2 KU
Exercise 2

Group 18

May 9, 2026

Institute of Software Engineering and Artificial Intelligence
Inffeldgasse 16b/II
8010 Graz

Nr.	Name	Matriculation number
1	Laura Maria Höber	12009439
1	Natalia Bocharova	12244471
2	Máté Hipságh	12445931

Contents

1 Part B: Answer-Set Programming

1.1 Task A

This part breaks down the placementA.asp Answer-Set Programming code. The first line described here corresponds to the first line in placementA.asp, and so on.

```
cell(R,C) :- row(R), col(C).
```

As required by the task description, the row(R) and col(C) variables are used, where R represents the R-th row and C the C-th column. Cell(R,C) is located at the intersection of R and C. cell(R,C) helps the program identify cells easier.

```
containWall(R,C) :- wall(R,C,_).
```

This variable helps to determine if a cell contains a wall. The _ symbol represents placeholder value, this means walls with different capacity are taken into account.

```
neighbouringWallCell(WR,WC,R,C) :- wall(WR,WC,_), cell(R,C),
not containWall(R,C),
R - WR <= 1, WR - R <= 1,
C - WC <= 1, WC - C <= 1.
```

Given a wall at row WR and column WC with any capacity. A cell at row R and column C is derivable if the absolute distance of that cell to the wall is at most 1 and that cell doesn't contain a wall. This rule is used in the program to determine all the cells around a wall.

```
placableCell(R,C) :- neighbouringWallCell(_,_,R,C).
```

This rule is used to determine if a lightbulb can be placed into a cell located at the intersection of R-th row and C-th column. The _ symbol in the neighbouringWallCell indicates that the walls at any place are taken into account.

```
{ placeLightBulb(R,C) } :- placableCell(R,C).
```

One of the most important parts of the program is described by this rule. Guess the placement of light bulbs in cells that are allowed to contain a lightbulb.

```
:- placeLightBulb(R,C), containWall(R,C).
:- placeLightBulb(R,C), not placableCell(R,C).
```

These constraints make sure that a lightbulb won't be placed on a cell that already contains a wall or is not directly adjacent to a wall.

```
obstructedRow(R,C1,C2) :- row(R), col(C1), col(C2), wall(R,WC,_), C1 < WC, WC < C2.
obstructedRow(R,C1,C2) :- row(R), col(C1), col(C2), wall(R,WC,_), C2 < WC, WC < C1.
```

These rules are used to determine if a wall is located on the R-th row. Given the C1-th and C2-th columns that span a section of row R. The first rule says that a wall is located inside or left to the section. The second rule says that a wall is located inside or to the right of the section. Both rules combined cover the whole row.

```
obstructedCol(C,R1,R2) :- col(C), row(R1), row(R2), wall(WR,C,_), R1 < WR, WR < R2.
obstructedCol(C,R1,R2) :- col(C), row(R1), row(R2), wall(WR,C,_), R2 < WR, WR < R1.
```

The same analogy can be used for columns as well

```
lighted(R,C) :- placeLightBulb(R,C).

lighted(R,C) :- cell(R,C), placeLightBulb(R,CB), C != CB, not obstructedRow(R,C,CB).

lighted(R,C) :- cell(R,C), placeLightBulb(RB,C), R != RB, not obstructedCol(C,R,RB).
```

The first rule states that the lightbulb lights up its own cell. The second rule describes that a cell at row R is lit up if there is a possible lightbulb placement in the same R-th row and the columns are different, and the horizontal path/row R doesn't contain a wall. For the third rule, the same analogy as for the second rule can be applied, but in this case for columns.

```
:- cell(R,C), not containWall(R,C), not lighted(R,C).
```

Following constraints remove all answer set, where a cell not containing a wall, isn't lit up.

```
#show placeLightBulb/2.
```

Causes the program to only print the possible location of the lightbulbs that satisfy all the rules and constraints.

Using an ASP solver, one unique result was found that satisfies all the constraints.

```
Answer: 1 (Time: 0.021s)
placeLightBulb(1,2) placeLightBulb(3,3) placeLightBulb(1,4) placeLightBulb(3,1)
```

1.2 Task B

In task B, the capacity of the walls had to be taken into account. placementB.asp contains the same rules as placementA.asp, but not the following code lines, which extend placementA.asp.

```
adjecentBulbs(WR,WC,N) :- wall(WR,WC,_),
N = #count { R,C : placeLightBulb(R,C), neighbouringWallCell(WR,WC,R,C) }.
```

Take a wall at the intersection of WR-th row and WC-th column. The neighbouringWallCell derives all possible adjacent cells of that wall, and placeLightBulb validates if a lightbulb is located at an adjacent cell. The N = #count represents the number of lightbulbs that are already placed around a wall.

```
:- wall(WR,WC,K), adjecentBulbs(WR,WC,N), N > K.
```

This constraint says that a wall is located at the intersection of the WR-th row and WC-th column with K capacity, and this wall has N lightbulbs around it, which is derived by the adjecentBulbs rule. If there is more lightbulbs around a wall than the capacity $\Leftrightarrow$ N $>$ K, then remove the answer set.

When taking the capacity of walls into account, we again obtained one unique result that satisfies all the constraints.

```
Answer: 1 (Time: 0.021s)
placeLightBulb(1,2) placeLightBulb(1,4) placeLightBulb(3,4) placeLightBulb(3,1)
placeLightBulb(4,3)
```

References